Faculté d' Informatique

Contraintes d'intégrités statiques & dynamiques

Master I-SSI

Dr. Ferrahi

Exercice 1

- Soit le schéma relationnel suivant :
  Personne (idpersonne, nom, datenaissance, lieunaissance, biographie)
  Film (idfilm, titre, année, description, idrealisateur*)
  RoleFilm (idacteur*, idfilm*, personnage)
  GenreFilm (idfilm, genre)

1. L'administrateur de la BD crée un utilisateur AGENT1 qui peut lire le contenu de la relation Film et un autre utilisateur AGENT 2 qui peut lire et modifier les relations Personne et Film, modifier le schéma de ces relations et transmettre ce pouvoir à d'autres agents. Donner les requêtes utilisées par l'administrateur pour répondre à ce besoin ?

- **CREATE USER** AGENT1 **IDENTIFIED BY** psw;
  GRANT SELECT ON FILM **TO** AGENT1 **WITH GRANT OPTION ;**

- CREATE USER AGENT2 IDENTIFIED BY psw;
  GRANT SELECT, UPDATE, INSERT, DELETE **ON** Personne **TO** AGENT2 **WITH GRANT OPTION** ;
  GRANT SELECT, UPDATE, INSERT, DELETE **ON** Film **TO** AGENT2 **WITH GRANT OPTION** ;

- Soit le schéma relationnel suivant :
  Personne (idpersonne, nom, datenaissance, lieunaissance, biographie)
  Film (idfilm, titre, année, description, idrealisateur*)
  RoleFilm (idacteur*, idfilm*, personnage)
  GenreFilm (idfilm, genre)

1. Ecrire une fonction PL/SQL prenant en paramètre le numéro d'une personne et retourne l'année du plus récent film réalisé par cette personne. La fonction doit retourner la valeur NULL si cette personne n'a réalisé aucun film.

```
CREATE OR REPLACE FUNCTION annee_dernier_film_realiser (p_idpersonne Personne.idpersonne%TYPE)
RETURN Film.annee%TYPE IS v_annee Film.annee%TYPE;
BEGIN
    SELECT MAX(F.annee) INTO v_annee
    FROM Film F
    WHERE F.idrealisateur = p_idpersonne;
    RETURN v_annee;
  EXCEPTION
     WHEN NO_DATA_FOUND THEN RETURN NULL;
END;
/
```


Un film ne peut pas avoir plus de cinq genres différents. Vérifier cette contrainte lors d'une insertion seulement.

```
CREATE OR REPLACE TRIGGER trigger_film_check_genre
BEFORE INSERT ON GenreFilm
FOR EACH ROW
 DECLARE
     v_nb_genres NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_nb_genres
    FROM GenreFilm WHERE idfilm = :NEW.idfilm;
    IF v_nb_genres >= 5 THEN
      RAISE_APPLICATION_ERROR(-20001, 'Le film ne peut pas avoir plus de cinq genres différents.');
    END IF;
END; /
```

Une personne ne peut pas jouer dans un film dont il/elle est le réalisateur. Vérifier cette contrainte seulement lors de l'insertion d'un nouveau rôle.

```
CREATE OR REPLACE TRIGGER trgger_rolefilm_check_realisateur
BEFORE INSERT ON RoleFilm
FOR EACH ROW
DECLARE
      v_idrealisateur       Film.idrealisateur%TYPE;
BEGIN
   SELECT idrealisateur INTO v_idrealisateur
   FROM Film
   WHERE idfilm = :NEW.idfilm;
   IF v_idrealisateur = :NEW.idacteur THEN
      RAISE_APPLICATION_ERROR(-20002, 'Une personne ne peut pas jouer dans un film dont elle est le réalisateur.');
   END IF;
END; /
```


L'année de sortie d'un film ne peut pas être supérieure à l'année courante. Vérifier cette contrainte lors d'une insertion et d'une update.

```
CREATE OR REPLACE TRIGGER trigger_film_check_year
 BEFORE INSERT OR UPDATE ON Film
FOR EACH ROW
DECLARE v_current_year NUMBER;
BEGIN
    SELECT TO_NUMBER(TO_CHAR(SYSDATE, 'YYYY')) INTO v_current_year
    FROM DUAL;
    IF :NEW.annee > v_current_year THEN
      RAISE_APPLICATION_ERROR(-20003, 'L''année de sortie du film ne peut pas être supérieure à l''année courante.');
   END IF;
END; /
```

Exercice 2

Exercice 1 :

Soit la base de données suivante :

- Employé (**Nump**, Nomp, SalaireP, PosteP, DateEmb, **Nserv***)
- Service (**Nserv**, NomServ, Localisation, DirecteurServ)
- Projet (**Nproj**, TitreProj, ResponsableProj, DateDeb, BudgetPro, **Nserv*, NB)**
- Affectation (**Nemp*, Nproj***, Date_Affec, Montant_indeminité)

1. Donner la requête qui permet d'ajouter le nombre d'employé participants à un projet.

ALTER TABLE projet **ADD** nb_emp NUMBER;

1. Exprimer la contrainte d'intégrité permettant de mettre à jour automatiquement ce nombre lors d'une affectation d'un employé à un projet ou lorsqu'il quitte un projet.

```
CREATE OR REPLACE TRIGGER  maj_nb_empl
AFTER INSERT OR DELETE ON Affectation
FOR EACH ROW
Begin
   IF inserting THEN
            ( UPDATE projet
              SET  nb_emp=nb_emp+1
              WHERE projet.Nproj=: new.Nproj);
   ELSE
```

```
IF deleting THEN
                (UPDATE projet
                 SET  nb_emp=nb_emp-1
                WHERE projet.Nproj= :old. Nproj);
ENDIF;
END IF;
END;
/
```


- Supposons que le SGBD sur lequel la BD est créé ne supporte pas les contraintes type « Clé Etrangère ».
- Proposez une solution permettant, lors de la création d'un projet dans un service de vérifier que ce numéro de service existe bien dans la table « Service ». Dans le cas contraire, l'opération d'ajout est rejetée.

En définition une Assertion.

En définition une Assertion.

- Assertion est une requête définie par l'admin
- Le trigger: se déclenche automatiquement suite à une modification

ASSERT asser_Q2 : **SELECT** Nserv **FROM** Projet
IS IN (**SELECT** Nserv **FROM** Service)

En utilisant un Trigger

```
CREATE OR REPLACE TRIGGER  Check_Foreign_Key
BEFORE INSERT ON Projet
FOR EACH ROW
 serv   Service.Nserv %TYPE;
Begin
    SELECT  Nserv INTO serv        serv /*variable dans laquelle je vais stocker le resulats
    FROM  Service
    WHERE nserv=:new.Nserv
```

```
  IF (Nserv IS NULL) THEN
        RAISE_APPLICATION.ERROR ('1250', Insertion impossible parceque le service
sque tu viens d'inserer au niveau de projet n'existe pas au niveau de la table service)
  END-IF
END

/* Avant de valider l'insertion dans la table Projet, on doit vérifier que le Nerv existe
dans la table Service

/* Si le service n'existe pas, la commande serv   Service.Nserv %TYPE;
Récupère la valeur NULL dans la variable serv
```


4. Ecrire une contrainte permettant de spécifier qu'un employé ne peut être affecté qu'à un projet du même service.

```
CREATE OR REPLACE TRIGGER
question4
BEFORE INSERT ON  Affectation
FOR EACH ROW
serv   empoloyé.Nserv %TYPE;

BEGIN
   SELECT E.nserv INTO srvr
  FROM Employé E, Projet P
  WHERE   E.Nserv= P.serv AND P.nproj = : new.Nproj and E.Numemp= :new.numemp
  IF (Nserv IS NULL) THEN
        RAISE_APPLICATION.ERROR ('20056', Insertion impossible
      car l'employe et le projet doivent faire partie du meme service)
  END-IF
END
```

5. Donner la requête permettant d'assurer que l'indemnité accordée à un employé lors de son affectation à un projet ne dépasse pas le 10% du budget du projet.

a. En définition une Assertion.

b. En définition un trigger.

Soit la base de données suivante :

Employé (**Nump**, Nomp, SalaireP, PosteP, DateEmb, **Nserv***)

Service (**Nserv**, NomServ, Localisation, DirecteurServ)

Projet (**Nproj**, TitreProj, ResponsableProj, DateDeb, BudgetPro, **Nserv*)**

Affectation (**Nemp*, Nproj***, Date_Affec, Montant_indeminité)

Solution avec assertion :

ASSERT CI-Ind ON Affectation A :

***Montant-Indemnité* < 10% (SELECT BudgetPro**

FROM Projet

WHERE Nproj = A. Nproj);

5. Donner la requête permettant d'assurer que l'indemnité accordée à un employé lors de son affectation à un projet ne dépasse pas le 10% du budget du projet.

a. En définition une Assertion.

b. En définition un trigger.

Solution avec les trigger :

CREATE OR REPLACE Trigger Verif-Budget
BEFORE INSERT ON Affectation
FOR EACH ROW
Bdg Projet.BudgetPro % Type

BEGIN
SELECT BudgetPro **INTO** Bdg
FROM Projet
WHERE Projet.Nproj =: NEW Affectation.Nproj;

IF (10% * Bdg <: (NEW. Montant_indemnité))
Then
Raise_Application_Error ('-20055, 'Insertion impossible l'indemnité accordée dépasse 10% du budget du projet');
END_IF
END.

Exercice 3

Exercice 2 :

Soit la base de données suivante :

Etudiant (**Mat**, Nom, Prénom, date_ naiss, Adr)

Enseignant (NSSEns, NomE, AdrE, Grade)

Module (**Cm**, NomM, Année, Filière)

Etud_Module (**Mat*, Cm***, note)

Ens_Mod (NSSEns*, Cm*)

1. Exprimer la requête qui permet d'afficher la liste des étudiants (Mat, Nom et prénom) dont la moyenne des notes des différents modules est supérieur à la moyenne générale des étudiants. (En définissant une vue).
2. On veut afficher la liste des enseignants par Année et par Filière
   a. Créer une vue permettant cette affichage.
   b. Peut-on mettre à jour cette vue ? Pourquoi ?
   c. Donnez les répercussions sur les catalogues.
3. On veut ajouter un attribut à la relation Module qui représente le nombre d'employé par Module.
   a. Donner la requête permettant cet ajout.
   b. Que proposez-vous pour mettre à jour automatiquement (ajout, suppression et modification) ce nombre.

1. Exprimer la requête qui permet d'afficher la liste des étudiants (Mat, Nom et prénom) dont la moyenne des notes des différents modules est supérieur à la moyenne générale des étudiants. (En définissant une vue).

Etudiant (Mat**, Nom, Prénom**, date_ naiss, Adr)
Enseignant (NSSEns, NomE, AdrE, Grade)
Module (Cm, NomM, Année, Filière)
Etud_Module (Mat*, Cm*, **note**)
Ens_Mod (NSSEns*, Cm*)

```
CREATE VIEW  Moyenne ( Matricule, Nom, Prenom, Moy)  AS
(  SELECT  Mat, Nom, Prenom, AVG (note)
   FROM Etudiant, Etud_Module
   WHERE  Etudiant.Mat= Etud_Module.Mat
  Group By Etudiant .Mat, Etudiant .Nom, Etudiant .Prenom)
```

a. Définition de la vue:

```
CREATE VIEW  Moyenne ( Matricule, Nom, Prenom, Moy)  AS
(  SELECT  Mat, Nom, Prenom, AVG (note)
   FROM Etudiant, Etud_Module
   WHERE  Etudiant.Mat= Etud_Module.Mat
  Group By Etudiant .Mat, Etudiant .Nom, Etudiant .Prenom)
```

→La vue permet de retourner la moyenne de chaque Etudiant

b. Utilisation de la vue:

```
SELECT  Matricule, Nom, Prenom
FROM Moyenne
Where Moy> (SELECT   AVG(Moy) From Moyenne);
```


2. On veut afficher la liste des enseignants par Année et par Filière

a. Créer une vue permettant cette affichage.

b. Peut-on mettre à jour cette vue ? Pourquoi ?

c. Donnez les répercussions sur les catalogues.

a. Créer une vue permettant cette affichage.

```
Create View List-Ens (Année, Filière, NomE ) As
(
   SELECT  Année, Filière, NomE
   FROM  Enseignant X, Ens-mod Y, Module Z
   WHERE X.NSSEns = Y.NSSEns and Y.CM= Z.CM
Group by Anné, Filière, NomE
);
```

2. On veut afficher la liste des enseignants par Année et par Filière

b. Peut-on mettre à jour cette vue ? Pourquoi ?

– on ne peut pas mettre à jour la vue car elle contient des jointures (sa mise à jour risque d'entrainer une incohérence par rapport aux tables de la base de données).

C. Répercussions sur les catalogues :

Insertion de la vue dans le catalogue Vues et ses attributs dans **le catalogue Attributs.**

- Dans le catalogue Relation on a :

– **Modification des cardinalités :**

Card(Vue) = card(Vue) +1

Card(Relation)= Card(Relation) +1

Card(Attributs)= Card(Attributs) + Nbre d'attributs de la vue

- **Modification des version, date de mise à jour**

21

3. On veut ajouter un attribut à la relation Module qui représente le nombre d'etudiants par Module.

b. Que proposez-vous pour mettre à jour automatiquement (ajout, suppression et modification) ce nombre.

Mise à jour automatique (ajout, modification, suppression) de ce nombre:

```
CREATE  TRIGGER Etud-Mod-Aj
AFTER INSERT ON  Etud_Module
FOR EACH ROW
(  UPDATE  Module
   SET  nbr-Etud = nbr-Etud+1
   WHERE  module.CM =: .NEW Etud_Module.CM
);
```

22

3. On veut ajouter un attribut à la relation Module qui représente le nombre d'employé par Module.

b. Que proposez-vous pour mettre à jour automatiquement (ajout, suppression et modification) ce nombre.

Mise à jour automatique (ajout, modification, suppression) de ce nombre:

```
CREATE  TRIGGER Etud-Mod-sup
AFTER DELETE ON  Etud_Module
FOR EACH ROW
(  UPDATE Module
   SET nbr-Etud = nbr-Etud-1
   WHERE Module.CM = :OLD Etud_Module.CM);
```



```
CREATE  TRIGGER Etud-Mod
AFTER INSERT OR DELETE ON  Etud_Module
FOR EACH ROW
BEGIN
IF Inserting THEN
  (   Update Module
      SET nbr-Etud = nbr-Etud+1
     WHERE CM = :NEW Etud_Module.CM);
ELSE
IF Deleting THEN
   (   Update Module
       SET nbr-Etud = nbr-Etud-1
       WHERE CM = :OLD Etud_Module.CM
   );
End_IF
End_IF
END
```